ROS 2 humble · TurtleBot3 line follower · 2026-10-11

Side experiment: Python vs C++ vision node

Project goal. The main project is a line follower in which Python and C++ work together to control one robot: a Python node (`rclpy`, OpenCV) detects the line in the camera image and publishes a steering error in [−1, 1], and a C++ node (`rclcpp`) turns that error into wheel speeds with the differential-drive equations $v_R = v + \omega d/2$, $v_L = v - \omega d/2$ and drives the robot. That system works end to end in simulation.

Why this benchmark exists. Comparing the two languages was not the goal. Once the Python + C++ system ran, a follow-up question came up: how much does writing the vision node in Python actually cost? To answer it, the vision node was also written in C++ with identical logic, and both versions were measured side by side. This report is that side experiment. Both versions crop the bottom 25% of a 640×480 frame, threshold blue in HSV, take the centroid with image moments and publish the same steering error.

PER-FRAME WORK, P50
3.0× faster
C++ 491 µs · Python 1,459 µs

CPU AT 30 FPS
4.3× less
C++ 2.9% · Python 12.2% of one core

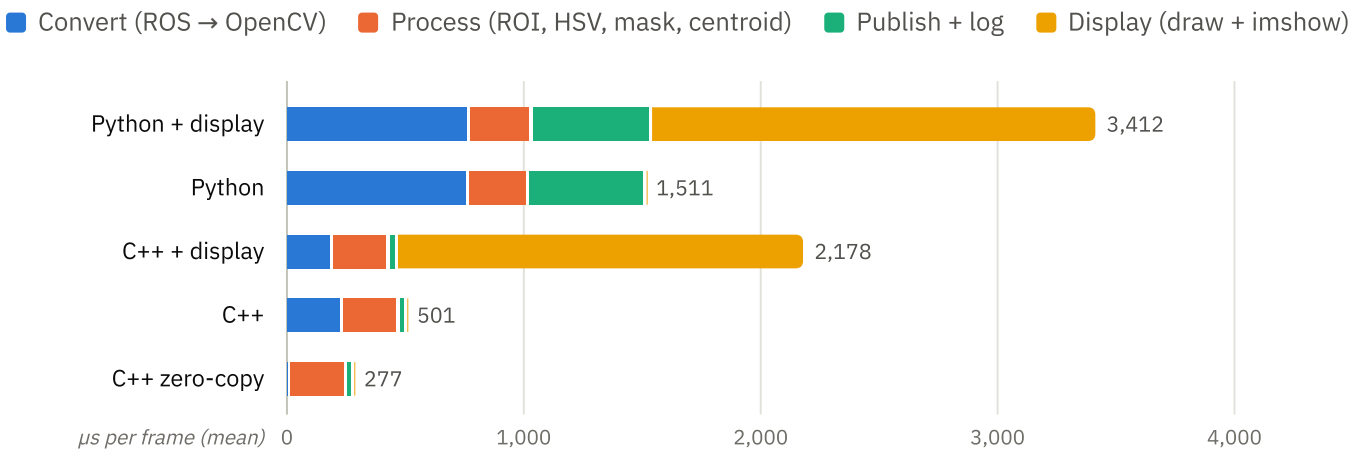
ROUND-TRIP LATENCY, P50
2.9× lower
C++ 1.13 ms · Python 3.33 ms

OUTPUTS IDENTICAL
Yes
492–498 frames compared per configuration, all equal

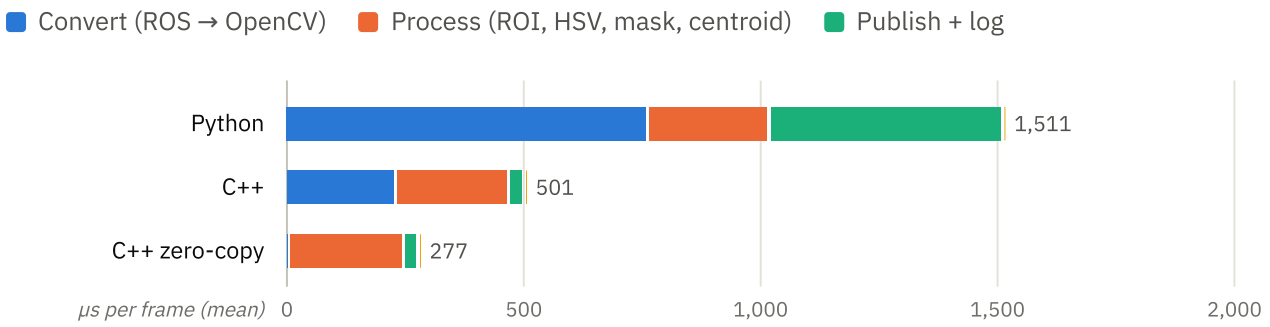
Where the time goes

Mean time per frame inside the node, split by stage. Each node timestamps its own callback with a monotonic clock (`perf_counter_ns` / `steady_clock`) at the same four points.

All configurations. Display dominates whenever it is on.



Display off. The work the robot actually needs.



With the debug windows on, drawing and `imshow` cost **1,879 µs** per frame in Python and **1,717 µs** in C++: more than all the line detection work combined. Display has to be off for a fair language comparison, and it is off on the robot.

The publish stage is **15×** slower in Python (494 µs vs 32.7 µs). It covers message serialization in `rclpy` and the throttled log call; in Python the f-string for that log line is built on every frame, even when the throttle suppresses it. This stage was not split further, so the share of each part is not measured.

Zero-copy (`toCvShare` instead of `toCvCopy`) cuts the C++ convert stage from **229 µs** to **4.27 µs**: the cost of copying one 640×480 frame (900 KB). Python has no equivalent; `imgmsg_to_cv2` always produces a new NumPy array.

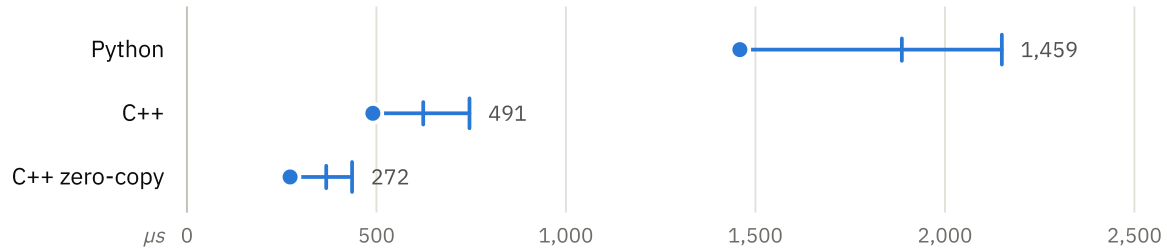
Configuration	Convert	Process	Publish + log	Display	Total
Python + display	768	262	504	1,879	3,412
Python	762	255	494	0.54	1,511
C++ + display	189	238	34.3	1,717	2,178
C++	229	239	32.7	0.18	501
C++ zero-copy	4.27	242	30.6	0.14	277

Mean µs per frame.

Per-frame work distribution

Percentiles of in-node time excluding display (convert + process + publish). Tail latency (p99) matters for control: one slow frame is one late steering command.

In-node work per frame. Dot = p50, short tick = p95, tall tick = p99.



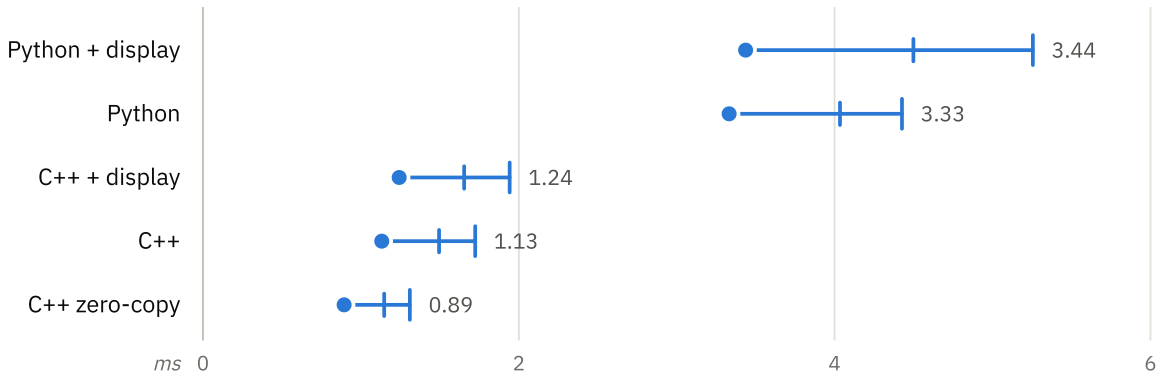
Configuration	p50 µs	p95 µs	p99 µs	p50 across reps	Max fps*	Frames
Python + display	1,491	1,872	2,097	1,478 – 1,498	293	3,273
Python	1,459	1,886	2,150	1,433 – 1,501	662	3,278
C++ + display	453	575	660	450 – 455	459	3,276
C++	491	624	746	487 – 500	1,996	3,275
C++ zero-copy	272	368	436	269 – 274	3,608	3,277

`* 1 / mean total callback time: the frame rate one core could sustain, ignoring transport.`

Round-trip latency

The harness publishes one frame, waits for the matching `/line_error`, then sends the next. This includes ROS 2 transport of the 900 KB image and the reply, so it is closer to what the robot experiences than in-node time alone. Display runs after the error is published, so it hardly moves this number; its cost shows up as CPU load and a lower maximum frame rate instead.

Frame published → error received. Dot = p50, short tick = p95, tall tick = p99.

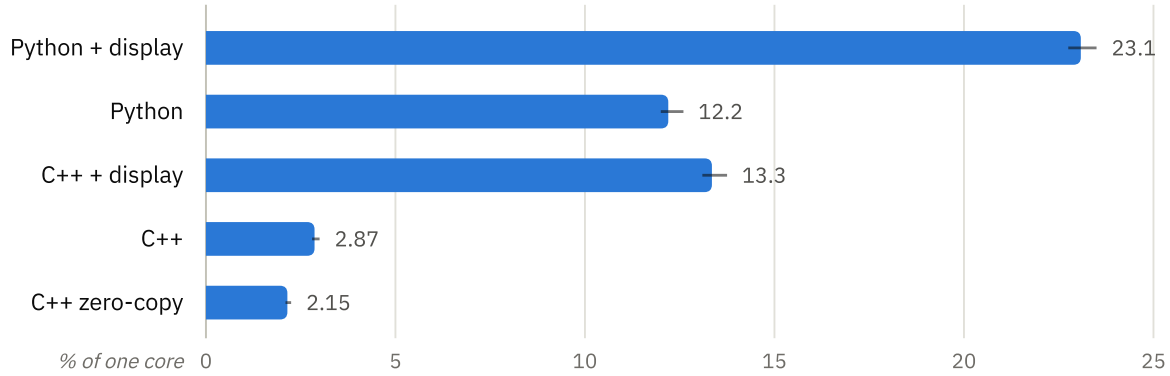


Configuration	p50 ms	p95 ms	p99 ms	Max ms	Samples	Lost
Python + display	3.44	4.50	5.26	7.14	1,489	11
Python	3.33	4.03	4.43	7.30	1,491	9
C++ + display	1.24	1.65	1.94	3.82	1,492	8
C++	1.13	1.50	1.72	2.16	1,492	8
C++ zero-copy	0.89	1.15	1.31	1.62	1,492	8

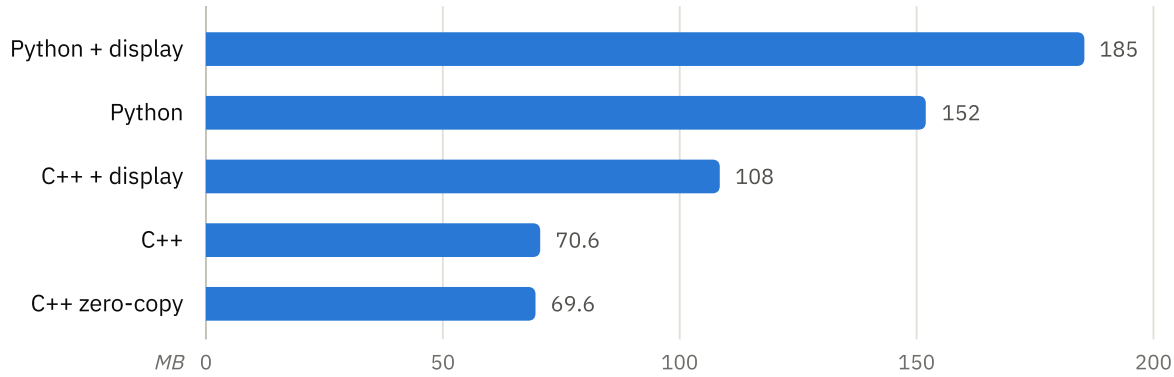
CPU and memory at camera rate

Frames streamed at 30 Hz (the simulated camera's rate) for 600 frames. CPU is the node process's user + system time over wall time, so 100% is one full core. Whiskers show the spread across repetitions.

CPU usage at 30 fps.



Resident memory (RSS).



Configuration	CPU % mean	CPU % range	RSS MB	Frames delivered
Python + display	23.1	22.7 – 23.5	185	99.4%
Python	12.2	12.0 – 12.6	152	99.4%
C++ + display	13.3	13.1 – 13.7	108	99.4%
C++	2.87	2.80 – 3.00	70.6	99.2%
C++ zero-copy	2.15	2.10 – 2.25	69.6	99.4%

`Frames delivered = /line_error replies ÷ frames sent during the 30 Hz stream (best-effort QoS, like the real camera topic).`

Do both versions compute the same thing?

A speed comparison only means something if the outputs match. During the latency test every configuration received the same frames in the same order; its `/line_error` values are compared with the Python node's.

Configuration	Equal values	Max |difference|	Result
Python + display	493 / 493	0.000000	**identical**
Python	498 / 498	0.000000	**identical**
C++ + display	492 / 492	0.000000	**identical**
C++	496 / 496	0.000000	**identical**
C++ zero-copy	493 / 493	0.000000	**identical**

Method

- **Configurations:** Python and C++, each with display on and off; C++ also with zero-copy conversion. C++ built with `CMAKE_BUILD_TYPE=Release`.
- **Input:** synthetic (300 frames, sim-like), published by the harness on `/camera/image_raw` with sensor-data QoS. Every configuration sees identical pixels.
- **Per run:** 60 warm-up frames (discarded), a 600-frame stream at 30 Hz (CPU, memory), then 500 lock-step frames (latency, outputs).
- **Repetitions:** 3 per configuration, interleaved (rep 1 of every configuration, then rep 2, ...) so slow drift affects all of them equally.
- **Isolation:** separate ROS domain, simulator not running, nodes started fresh for every run.

Limitations

- Frames are synthetic; the processing cost does not depend much on image content (every pixel of the ROI is converted and thresholded either way), but real camera frames should be used to confirm.

- One machine, laptop CPU with frequency scaling (governor: `powersave`). A lightly loaded core runs at a lower clock, so a configuration that does less work can look slightly slower per stage than one that keeps the core busy (compare C++ with and without display). Absolute numbers will differ on the robot's computer; the ratios are the transferable result.
- The harness is written in Python; its own overhead is part of every round-trip number, equally for all configurations.
- OpenCV does the heavy lifting in both languages (the Python `cv2` calls run the same C++ library), so this measures binding, conversion and interpreter overhead around identical kernels, not "Python vs C++" for hand-written loops.

Environment

CPU	AMD Ryzen 7 5800H with Radeon Graphics (8 cores / 16 threads)
RAM	22.8 GB
CPU governor	powersave
On AC power	yes
OS / kernel	Ubuntu 22.04.5 LTS / 6.8.0-138-generic
ROS 2	humble, rmw_fastrtps_cpp (default)
Python / NumPy	3.10.12 / 1.21.5
OpenCV (Python / C++)	4.5.4 / 4.5.4
Compiler / build type	g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0 / Release
Frames	synthetic (300 frames, sim-like)
Run date	2026-10-11T12:19:08

Code version and tooling

Measured on **version 2** of the vision nodes (git tag `v2`), which adds the per-stage timing, the `show_debug` switch and the C++ `zero_copy` option to the original implementation. With default parameters it behaves exactly like version 1. The C++ node was built with `-DCMAKE_BUILD_TYPE=Release`.

Tooling not included. The measurement harness and the report generator are not part of the repository; only this report is.